Daily Paper Review: LatentLoop: Scaling Looped MoE Models with Asymmetric Compute Allocation

Auto-generated Report

October 17, 2026

1 Paper Summary

1.1 Problem and Motivation

Scaling large language models demands proportional increases in parameters, memory, and compute. Two orthogonal efficiency paradigms—Mixture-of-Experts (MoE, sparse computation) and Looped Transformers (parameter reuse across iterations)—have been combined in prior work (SMELT), but the combination is *naïve*: every loop iteration receives identical computational treatment despite sharing the same parameters. He et al. (arXiv:2610.19975) discover a fundamental **asymmetry** in how different loop iterations contribute to model output and exploit it to achieve superior efficiency.

The authors attach learnable per-visit residual scaling coefficients α_v to a standard looped MoE model and observe a striking pattern: the first visit through the shared middle block consistently learns coefficients roughly $3\times$ smaller than the second visit. At 1.3B parameters, $(\alpha_1, \alpha_2) \approx (0.27, 0.87)$; at 3B, $(\alpha_1, \alpha_2) \approx (0.36, 1.09)$. This pattern persists with 3 visits (ratios $\approx 2.4\times$ between consecutive coefficients) and 4 visits (ratios $\approx 2.0\times$), with the first visit always contributing the smallest residual updates and the last the largest. The implication: *not all loop iterations deserve equal compute*, and allocating uniform resources across visits wastes computation on early iterations that contribute less.

Existing approaches fail to exploit this: vanilla MoE requires linear parameter growth with no reuse; vanilla looped transformers treat all iterations symmetrically; and SMELT (the prior looped MoE method) treats all visits equally while exhibiting budget-matching deviations up to 3.9% in FLOPs-per-token, making fair comparisons questionable.

1.2 Method

LatentLoop exploits the discovered asymmetry by **compressing the token sequence during the first (less important) visit** and running the second visit at full resolution, freeing compute that is reinvested into increased model width.

Base Looped Architecture. The transformer's L stored layers are divided into three blocks: an entry block (first $L/4$ layers, executed once), a middle block (next $L/2$ layers, executed **twice** with shared parameters), and an exit block (final $L/4$ layers, executed once). This yields $3L/2$ total layer executions from L stored layers.

Asymmetric Compute via Sequence Compression. The LatentLoop pipeline applies a compression–process–decompress–process sequence:

$$Z = \mathrm{Enc}(H), \quad Z \in \mathbb{R}^{l \times d} \quad \text{(compress: } n \to l = n/k \text{ tokens)} \tag{1}$$

$$\Delta = F_1(Z) - Z \quad \text{(first visit on compressed sequence)} \tag{2}$$

$$\tilde{H} = H + \mathrm{Dec}(\Delta) \quad \text{(decompress residual and add back)} \tag{3}$$

$$Y = F_2(\tilde{H}) \quad \text{(second visit at full resolution)} \tag{4}$$

where k is the compression factor (k=2 in all experiments, halving the sequence for the first visit).

Encoder and Decoder Design. The encoder uses **mean aggregation**, simply averaging adjacent token pairs:

$$Z_j = \frac{H_{2j-1} + H_{2j}}{2}, \quad 1 \le j \le l \tag{5}$$

This is preferred over a learnable causal convolution alternative for simplicity and competitive performance. The decoder uses a **Scaled Decoder (SD)** with learnable per-channel writeback coefficients:

$$\mathrm{Dec}(\Delta)_{kj+s} = w_s \odot \Delta_j, \quad 0 \le s < k \tag{6}$$

where $w_s \in \mathbb{R}^d$ are element-wise scaling vectors ($2d$ total parameters for k=2). Analysis of learned coefficients reveals the nearer receiver position learns a higher mean weight (≈ 0.199) than the farther position (≈ 0.160), confirming that the latent update is more reliable for closer original positions.

MoE Integration. A critical architectural finding: MoE routers and experts should be **shared** across both visits, while attention blocks should be **untied** (independent per visit). Ablations show that untying MoE experts *hurts* performance (NLL 2.0340 vs. 2.0199), while untying attention improves it. The expert count is reduced by 1 from the baseline allocation to compensate for the additional attention parameters within the parameter budget. All models use 8 query heads, 2 KV groups, top-8 expert routing, and 4096 context length.

Budget Matching. To ensure fair comparison, four resources are matched sequentially: (1) adjust attention projections for KV cache size, (2) adjust model width d for per-step FLOPs, (3) adjust stored expert count E for total parameters, (4) choose training steps for total training FLOPs. Maximum deviations: 0.6% (parameters), 0.8% (KV cache), 0.9% (FLOPs/token)—substantially tighter than SMELT's 1.0%, 3.6%, 3.9%.

Which Visit to Compress. Ablations confirm the asymmetry is directional: compressing the first visit and running the second at full resolution is the *only* configuration that improves over the baseline loop (NLL 2.0199 vs. 2.0293). Compressing the second visit degrades performance (2.0386), and compressing both is worst (2.0467).

1.3 Results

Experiments span 1.3B, 3B, and 7B total parameters (with ∼1B active parameters at 7B), trained on FineWeb-Edu with 40B–140B tokens.

Scaling Law Analysis: At matched NLL = 1.988, LatentLoop achieves **19.0% training FLOPs reduction** vs. vanilla MoE Base and **7.9% training FLOPs reduction** vs. MoE Loop.

Downstream Benchmarks (9 tasks): LatentLoop achieves the highest average across all 6 settings (3 sizes × 2 shot settings). At 7B/140B tokens, 5-shot average: **63.40** (LatentLoop) vs. 62.55 (MoE Loop) vs. 62.19 (MoE Base). At 3B/70B: **58.51** vs. 57.33 vs. 56.69.

Hardware Efficiency: Despite matched theoretical FLOPs, LatentLoop achieves **9% wall-clock speedup at 3B** and **13% at 7B** over MoE Loop, because freed FLOPs reinvested into wider d improve GPU hardware utilization through larger matrix operations.

Compute Reduction Ablation (3B/40B): Sequence compression (NLL 2.0199) outperforms all alternative first-visit reduction strategies: reducing residual width (2.0361), expert intermediate width (2.0364), active expert count (2.0349), and stride-based sequence reduction (2.0303). Sequence compression preserves full model width and expert activation per latent position.

Model Width Reallocation: LatentLoop models are consistently wider than MoE Loop models at matched budgets. At 7B: d=1696 (LatentLoop) vs. d=1552 (MoE Loop) vs. d=1856 (MoE Base), with 76 vs. 93 vs. 64 experts respectively.

1.4 Limitations

(1) **Incomplete mechanistic understanding**: The authors explicitly acknowledge that why the first visit consistently learns smaller coefficients remains an open question—the asymmetry is empirically robust but theoretically unexplained. (2) **Two-visit restriction**: While multi-visit coefficient analysis extends to 3–4 visits, the full LatentLoop architecture with compression is only evaluated with 2 visits. (3) **No inference efficiency analysis**: The paper focuses entirely on training FLOPs and validation loss; inference latency, memory footprint during generation, and throughput are not discussed, despite the looped architecture inherently increasing inference latency (the middle block executes twice sequentially). (4) **Scale ceiling at 7B**: All experiments cap at 7B total parameters (∼1B active); whether the asymmetric compute advantage persists at 70B+ scales is unknown. (5) **Fixed compression factor**: Only k=2 is evaluated; the interaction between compression ratio and model size or visit count is unexplored. (6) **Single dataset**: All training uses FineWeb-Edu; generalization to code, multilingual, or instruction-tuning data is untested. (7) **Fixed block split**: The $L/4$–$L/2$–$L/4$ partitioning is inherited from SMELT without exploration of alternatives.

1.5 Relevance to Research

LatentLoop directly extends the looped transformer line of work central to Topic T1 (LLM efficiency). The Iso-Depth scaling laws (Review #42) established that one recurrence of a looped transformer is worth approximately 0.6–0.8 of a unique layer in terms of effective depth, but treated all recurrences as equivalent—LatentLoop reveals that this "exchange rate" varies *across visits*, with the first visit contributing less than subsequent ones. The Recurrent Looped Transformer (Review #121) introduced input-dependent recurrence depth and progressive state refinement; LatentLoop provides the complementary insight that even with fixed recurrence depth, the *compute intensity* per visit should vary. The MoE integration connects to ELDR (Review #85), which optimized expert-locality-aware decode routing for disaggregated MoE serving—LatentLoop's finding that shared experts across visits outperform untied experts adds a new constraint for MoE routing design: expert sharing enables cross-visit knowledge transfer that untying destroys. The hardware efficiency gain (9–13% wall-clock speedup from wider matrices) resonates with FreeToken (Review #116), which similarly exploited hardware-aware execution planning for MoE models; LatentLoop achieves a similar effect through architectural design rather than runtime optimization. The quantization compatibility of looped architectures (fewer unique parameters) connects to TRACE (Review #120), which applied FP4 quantization to MoE RL training—LatentLoop's parameter-sharing structure naturally amplifies quantization benefits since the shared middle block's parameters need be quantized only once despite

executing twice.

2 Follow-up Research Directions

2.1 Direction 1: Asymmetric Looped Decoding for Diffusion Language Models

Gap: LatentLoop's sequence compression operates during *training* on the forward pass through a looped encoder-style model, but diffusion LLMs (dLLMs) perform iterative *denoising* where each step refines the entire sequence. Current dLLM architectures like Sumi (Review #76) apply identical computation at every denoising step, despite evidence (from Subliminal Clocks, Review #99) that early denoising steps perform qualitatively different work (semantic layout) than late steps (token refinement). No existing method applies asymmetric compute allocation across denoising iterations in dLLMs.

Approach: Extend LatentLoop's looped architecture to dLLM denoising by treating early denoising steps as "first visits" and late steps as "second visits." Specifically, partition the T denoising steps into two phases: the first $T/2$ steps run on a **compressed latent sequence** (using LatentLoop's mean-aggregation encoder, k=2), while the final $T/2$ steps operate at full resolution. The middle transformer block is shared across all denoising steps (looped), but the compression/decompression applies only to early steps. This directly mirrors LatentLoop's finding that early visits contribute smaller residual updates and can tolerate reduced resolution. The decompression uses the scaled decoder to restore full-length sequences before the high-fidelity late steps refine individual tokens. For MoE-based dLLMs, the shared expert design from LatentLoop applies naturally—the same experts process tokens across denoising steps, enabling cross-step knowledge sharing. This combines LatentLoop's asymmetric compute (T1) with dLLM architecture (T4), potentially delivering the throughput benefits of SpecFold (Review #122) through architectural design rather than speculative decoding.

Feasibility: Medium-high. The mean-aggregation encoder and scaled decoder are simple operations that can be integrated into existing dLLM forward passes. The main challenge is that dLLM denoising involves bidirectional attention, and mean-pooling adjacent tokens may lose critical positional information for mask prediction. A position-preserving variant (e.g., pooling semantically similar rather than adjacent tokens) may be needed. SoloQ's W4A4 quantization (Review #123) could compound the efficiency gains.

2.2 Direction 2: Visit-Adaptive Expert Routing for Multi-Visit Looped MoE

Gap: LatentLoop shares MoE experts across visits and finds this outperforms untied experts. However, the *router* is also shared, meaning both visits route tokens to the same experts using the same scoring function. The learned residual coefficient asymmetry ($\alpha_1 \ll \alpha_2$) suggests that the two visits perform qualitatively different computation (coarse structure vs. fine refinement), yet the routing strategy is visit-agnostic. No existing method adapts expert routing based on which loop iteration is executing.

Approach: Introduce **visit-conditioned routing** that shares experts but uses visit-specific routing functions:

$$g_e^{(v)}(x) = \text{softmax}\big(W_r x + b_v\big)_e, \quad v \in \{1, 2\}$$

where W_r is the shared routing matrix and $b_v \in \mathbb{R}^E$ is a lightweight per-visit expert bias (E parameters per visit). The bias allows the router to systematically prefer different expert subsets for different visits without untying the full routing function. This is motivated by the finding in ELDR (Review #85) that routing decisions critically affect MoE serving efficiency, and by LatentLoop's own observation that shared experts outperform untied ones—visit-conditioned

routing preserves sharing while allowing specialization. The per-visit bias can be initialized to zero (recovering the shared baseline) and learned during training. Analysis of learned biases would reveal whether the model naturally develops visit-specialized expert usage patterns, providing mechanistic insight into the asymmetry that LatentLoop leaves unexplained. Connect to Ring-Zero (Review #96), which scaled MoE RL training to a trillion parameters—visit-adaptive routing could improve the efficiency of MoE scaling in looped architectures at extreme scales.

Feasibility: High. The modification adds only $2E$ parameters (negligible) and a single bias addition to the routing function. Training is unchanged. The main risk is that visit-conditioned routing may fragment expert utilization, reducing the effective expert pool per visit. Load-balancing losses should be computed per-visit to prevent this.

2.3 Direction 3: Graduated Compression for Continual Consolidation in Looped Agents

Gap: LatentLoop demonstrates that sequence compression during less-important loop iterations improves training efficiency, but applies a fixed compression ratio (k=2) throughout training. Cortex (Review #126) shows that continual memory consolidation in agents requires sequential updates where earlier parameters are frozen and new structural expansions handle new knowledge. No existing method combines looped architectures with continual learning, despite the natural compatibility: the looped middle block's shared parameters could serve as a stable "core memory" while visit-specific components (LatentLoop's untied attention blocks) could absorb new knowledge.

Approach: Design a **continual looped agent** where the shared middle block (MoE experts + router) is the frozen parametric core, and visit-specific attention blocks are the continual learning substrate. For each new consolidation episode (following Cortex's context-to-memory distillation framework), add a small structural expansion to the second-visit attention block only—motivated by LatentLoop's finding that the second visit contributes more and thus is the appropriate locus for new knowledge. The first-visit compression ratio k is **graduated**: it starts at k=2 (LatentLoop's default) and increases as more consolidations accumulate (e.g., k=3 after 20 consolidations, k=4 after 50), progressively reducing the first visit's compute budget as the core memory stabilizes and the second visit's expanded attention absorbs more task-specific knowledge. This connects LatentLoop's asymmetric compute discovery (T1) with Cortex's structural plasticity for continual learning (T3): the architecture naturally separates stable knowledge (shared middle block, first visit for coarse processing) from evolving knowledge (second-visit attention expansions). The query replay mechanism from Cortex would operate on the full-resolution second visit only, avoiding wasteful rehearsal through the compressed first visit.

Feasibility: Medium. The looped architecture's natural separation between shared and visit-specific components aligns well with continual learning's need for stable vs. plastic components. The main challenge is that LatentLoop's budget-matching methodology assumes fixed architecture throughout training; graduated compression requires re-deriving width allocations at each compression transition. The scaling laws from Iso-Depth (Review #42) could inform how to adjust width as k changes while maintaining effective depth.

3 Conclusion

LatentLoop establishes a foundational principle for looped transformer architectures: *loop iterations are not created equal*. By discovering that early visits through shared parameters consistently learn smaller residual coefficients (approximately 3× smaller than later visits), the paper motivates asymmetric compute allocation—compressing the token sequence during the first visit and running the second at full resolution. This simple architectural change, com-

bined with mean-aggregation encoding and learned writeback coefficients, achieves 19% training FLOPs reduction over vanilla MoE and 7.9% over standard MoE Loop, with consistent downstream improvements across 1.3B–7B scales. The finding that MoE experts should be shared across visits while attention blocks should be untied provides a nuanced design principle for looped MoE architectures: expert knowledge benefits from cross-visit transfer, but attention patterns need visit-specific specialization. The hardware efficiency bonus (9–13% wall-clock speedup from reinvesting freed compute into wider matrices) demonstrates that theoretical FLOPs savings can translate to practical speedups when the architectural design aligns with GPU execution characteristics. For LLM efficiency research, LatentLoop refines the looped transformer paradigm from "how many recurrences?" to "how much compute per recurrence?", opening a new axis of optimization complementary to quantization, pruning, and distillation.

References

[1] Z. He, Y. Chai, X. Chen, T. Zhu, Z. Wu, Y. Luo, Y. Li, and Y. Cheng. LatentLoop: Scaling Looped MoE Models with Asymmetric Compute Allocation. *arXiv preprint arXiv:2610.19975*, 2026.

[2] Y. Cheng et al. SMELT: Scaling Looped MoE Transformers. *arXiv preprint*, 2026.

[3] J. Author et al. How Much Is One Recurrence Worth? Iso-Depth Scaling Laws for Looped Language Models. *arXiv preprint arXiv:2604.21106*, 2026.

[4] Z. Author et al. Recurrent Looped Transformer. *arXiv preprint arXiv:2610.07591*, 2026.

[5] J. Zhang et al. Cortex: Consolidating Context into Parametric Memory. *arXiv preprint arXiv:2610.20399*, 2026.